

Timed Supply — Planning Package

Timed Supply Control System — Planning Package

Working name · Draft for sign-off · 2026-07-09

A greenfield product modelled on the VoltLync EV-CSMS architecture, for a generic on/off supply billed by time. Device hardware + firmware are delivered by separate teams; our scope is the server, the apps, and the device WS protocol. This package contains the full **Project Plan** (clarifying questions, task breakdown, estimate), the **2-Week POC** scope, the **glossary** (CONTEXT), and the load-bearing decision record (**ADR-0001**).

Timed Supply Control System — Project Plan (v1)

Status: draft for sign-off · **Date:** 2026-07-09 · Working name only.

A greenfield product: customers find a **Machine** on a map (or scan its QR), start it from a native app, it delivers a **safe low-voltage supply** while on, and they are billed for the **connected-on seconds**, deducted from a prepaid **Wallet**. Unattended, single-operator, pure-time (no flow sensor). **Device hardware and firmware are delivered by separate teams** — our scope is the server, the apps, and the device WS protocol we hand them as the interface contract.

The VoltLync EV-CSMS in this repo is the **reference architecture** — we reuse its *patterns* (WebSocket device link, event-sourced wallet, Redis liveness registry, Clerk auth, Capacitor app, Next.js admin) but write fresh code. See [CONTEXT.md](#) for the glossary and [adr/0001-connectivity-gated-server-metered-supply.md](#) for the one load-bearing decision.

0. Decisions already locked

#	Decision	Rationale
D1	Greenfield , EV code is reference only	Different billing model
D2	Server-metered wall-clock time , no device counter	Time has no meter (ADR-0001)
D3	Connectivity-gated supply — relay opens on link loss	Closes the offline-unbilled-supply safety hole (ADR-0001)
D4	Bill only connected-on seconds ; off-gaps unbilled (exact)	Supply is physically off when disconnected
D5	30-min resume window ; auto-re-energize on reconnect	Safe: supply is safe-low-voltage + unattended
D6	Pure-time, no sensor → <i>zero-supply blindness accepted</i>	See Q-C3
D7	Wallet-only funding; QR = machine-selector, not guest-pay	Simplest v1
D8	Single operator (all machines owned) → no settlement layer	Removes ~30% of EV scope
D9	Native Capacitor customer app + Next.js admin + Clerk	Wireframe demands native; reuse proven stack
D10	Hardware + firmware owned by separate teams ; we own the device WS protocol as the interface	Out of our workstream; we deliver the contract they build to
D11	Pricing is per-machine, controllable from the admin UI	Operator tunes rates without a deploy
D12	Defer : QR/UPI-guest, formal invoicing, OCPI/external feed	Not needed for v1

1. Clarifying questions

Grouped by how much they move the design. **Bold = blocks the estimate or the architecture.**

A. Device interface (coordinate with the hardware/firmware team)

- **A1. Confirm the firmware honours the ADR-0001 contract:** relay + heartbeat + a **watchdog that opens the relay on link loss** + remote start/stop. The whole billing model rests on this.
- A2. **Heartbeat interval?** Sets metering granularity + how fast a disconnect is detected. (*Propose ~15–30s.*)
- A3. **Watchdog "open-on-link-loss" timeout** — seconds of no server ping before the relay opens. (*Propose ~10s; must be < heartbeat × 2.*)
- A4. Agree the **device WS protocol** shape — custom JSON-over-WSS (connect, heartbeat, start, stop, status). (*We propose; they implement.*)
- A5. **Provisioning** at manufacture: auth key + fixed GPS coords + QR sticker loaded onto each unit.

B. Billing / pricing

- **B1. Rate unit — per second or per minute?** Drives the ticker cadence and how the rate is stored/displayed. (*Recommend: meter internally in **seconds**, admin enters ₹/min, bill seconds × rate/60.*) Couples with Q-D1.
- B2. Confirm **per-machine rate set in admin** (D11). Any **global default**, and a **flat start fee / free grace seconds / minimum charge / rounding rule?** (*Recommend: per-machine rate, no minimum, round to the paisa at stop.*)
- B3. **Max session duration / safety ceiling** independent of wallet budget?
- B4. Overshoot: a customer can exceed budget by a few seconds between ticks → wallet dips slightly negative. Clamp, absorb, or block? (*Recommend absorb + observe, mirroring the EV negative-balance posture.*)

C. Product / supply

- **C1. What is the supply, concretely?** Needed for app copy, the "picture of machine" screen, safety labelling, and to confirm the safe-low-voltage + unattended envelope holds for **every** machine (ADR-0001 depends on it).
- C2. Is a session ever controllable from a **physical button on the machine**, or strictly app-only? (*Unattended ⇒ recommend app-only.*)
- **C3. Accept zero-supply blindness?** With no sensor, a customer pays for on-time even if the thing behind the relay was dead. Accept, or fund a flow sensor (changes the architecture toward device-metering)?

D. Usage & scale (drives the ticker design and capacity)

- **D1. What is the average session length?** Seconds, minutes, hours? Sets the right rate unit, the ticker cadence, and realistic wallet/top-up sizes.
- D2. Expected **sessions per day per machine**, and **peak concurrent sessions** across the fleet? (*Drives the per-session ticker design — Task 2.3.*)
- D3. How many **machines** at launch and at 12 months?
- D4. **iOS + Android both at launch?** Apple/Google developer accounts ready?
- D5. Launch **geography, currency, timezone, languages** (IST assumed per EV precedent).

2. Task breakdown

Six workstreams. Backend and clients run largely in parallel once the device WS protocol (2.7) and the API contract (3.4) are frozen. Firmware is delivered by a separate team against protocol 2.7.

WS1 — Foundations

- 1.1 Repo scaffold, Docker compose (postgres, redis, backend), env plumbing, CI, DB SSL pattern (reuse EV `db_ssl.py`)
- 1.2 Data model + Aerich migrations: `Machine`, `SupplySession`, `User`, `Wallet`, `WalletTransaction`, `Tariff`, `Receipt`, `MachineAuthKey`
- 1.3 Clerk integration (backend), logging, monitoring baseline

WS2 — Device link & session engine (*novel, highest-risk core*)

- 2.1 Device WebSocket endpoint + connection manager (in-memory dict + Redis liveness mirror) — pattern from `connection_manager.py`
- 2.2 Per-Machine auth on WS upgrade (pattern from ADR-0020) + heartbeat + status
- 2.3 **Server-side per-session ticker** — accrues connected-on seconds, enforces the **max-on cap**, fires stop. *Novel vs EV (server owns the clock)*.
- 2.4 Session lifecycle: start (snapshot budget → `max_on_seconds`), accrue, stop (finalize + bill), remote start/stop dispatch with ack
- 2.5 Disconnect → suspend → **30-min resume** → finalize-past-window; startup invariant check `resume_window >> heartbeat`
- 2.6 **Device simulator** (headless) for integration tests
- 2.7 **Freeze + document the device WS protocol** → handoff to the firmware team

WS3 — Wallet, payments & billing

- 3.1 Event-sourced wallet ledger (non-negative amounts, type-directed, SUM-derived balance + Redis cache) — pattern from `wallet_service.py`
- 3.2 Razorpay top-up: create order, verify, webhook, idempotency
- 3.3 Billing at stop: `rate × billable_seconds`, wallet debit, **Receipt** (simple v1; formal invoicing out of scope)
- 3.4 **Freeze the customer + admin REST API contract** (unblocks WS4/WS5)

WS4 — Customer native app (Capacitor + Vite + React + Clerk)

- 4.1 Scaffold + Clerk + API client + native plugins (barcode, Razorpay, geolocation)
- 4.2 Map/Home (machine search + markers) · machine-QR Scanner
- 4.3 Machine detail (picture + START/STOP + **live elapsed + running cost**)
- 4.4 Sessions history + Receipt view
- 4.5 Profile + Wallet top-up
- 4.6 App-store builds & submission (iOS + Android)

WS5 — Admin web (Next.js + Clerk)

- 5.1 Scaffold + Clerk middleware
- 5.2 Machines: list, detail, **provision** (auth-key, coords, QR sticker), liveness
- 5.3 **Rate/pricing config (per-machine, editable in UI — D11)** · Sessions console · Users

WS6 — Infra, hardening, UAT

- 6.1 Staging + prod compose, nginx, managed postgres/redis, deploy pipeline
- 6.2 Backend test suite (lifecycle, billing, ledger, **disconnect/resume**, cap)
- 6.3 Edge-case & load testing via simulator · security review · integration UAT with the firmware team's real units

3. Time estimate

Assumptions: ~2 engineers who know the reference architecture (their own). Firmware/hardware are **not** in this estimate (separate teams). Estimates are **engineer-weeks (EW)**; calendar assumes parallelism after the device protocol (2.7) and API contract (3.4) freeze.

Workstream	EW	Notes
WS1 Foundations	2	Scaffold, data model, CI, auth
WS2 Device link & session engine	5	The hard part; 2.3/2.5 are novel
WS3 Wallet, payments & billing	3	Ledger + Razorpay + billing/receipt
WS4 Customer native app	4	All screens + native + app-store
WS5 Admin web	2	Lean, single-operator, editable pricing
WS6 Infra, hardening, UAT	3	Deploy + tests + integration UAT
Total	≈19 EW	

Calendar rollups:

- **2 engineers, partial parallelism** → **≈ 11–13 weeks (~3 months) to production v1.**
- 1 engineer, serial → **≈ 19–22 weeks (~4.5–5 months).**
- Add **+2–3 weeks** app-store review buffer (Apple, especially).

Phasing (2-engineer track):

1. **Weeks 1–2** — WS1 + freeze device protocol (2.7, → firmware team) and API contract (3.4).
2. **Weeks 2–6** — WS2 core (session engine + simulator) || WS3 wallet start.
3. **Weeks 4–9** — WS4 app || WS5 admin against the frozen contract.
4. **Weeks 8–11** — billing close-out, WS6 hardening + UAT.
5. **Weeks 11–13** — deploy, app-store submission, launch buffer.

Risks that move the number

- **Integration dependency:** the firmware team must deliver units that honour protocol 2.7 (esp. the watchdog) on time; our simulator de-risks *our* side, not theirs. Slippage there delays field UAT (WS6.3), not our build.
- ↑ **+2–4 EW** if the supply turns out **not** uniformly safe-low-voltage / attended (Q-C1) — protocol changes for interlocks + a different resume policy.
- ↑ **+3–5 EW** if a **flow sensor** is funded to kill zero-supply blindness (Q-C3) — reopens the device-metering architecture and the protocol.
- ↓ **-2–3 EW** if a **mobile-web PWA** replaces the native app (drops app-store + native plugins) — at the cost of camera/QR polish and store presence.

Appendix — reference-architecture mapping

New component	Borrow the <i>pattern</i> from
Device WS + connection manager	<code>backend/core/connection_manager.py</code> , <code>routers/ocpp_ws.py</code>
Per-Machine WS auth	ADR-0020 (Charger Auth Key)
Wallet ledger	<code>services/wallet_service.py</code> (event-sourced, SUM-derived)
Budget cap → auto-stop	<code>services/wallet_session_service.py</code> (re-timed to seconds)
Disconnect/resume/staleness	<code>services/transaction_finalizer.py</code> , ADR-0022
Razorpay top-up	<code>routers/wallet_payments.py</code> , <code>services/razorpay_service.py</code>
Native customer app	<code>app/</code> (Capacitor + Vite + React + Clerk)
Admin web	<code>frontend/</code> (Next.js + Clerk)
Deploy / env plumbing	<code>root docker-compose*.yaml</code> , <code>CLAUDE.md</code> <code>env-var</code> checklist

Timed Supply — 2-Week POC

Goal: prove the *one uncertain thing* — the **connectivity-gated, server-metered session loop** (ADR-0001) — end-to-end, **switching a real relay**. Everything else is faked or cut. Throwaway scaffolding that becomes the v1 skeleton, not v1 itself.

What the POC must prove (success criteria = the demo script)

A single clickable demo, with a **physical relay** (driving an LED or a small appliance) as the Machine:

1. The relay device connects and heartbeats → app shows "available".
2. Wallet shows a seeded balance (₹100). Tap **Start** → **the relay physically closes** (supply on), **live timer + running cost** tick up.
3. Tap **Stop** → **relay opens**, session ends, wallet **debited** by `rate × seconds`, receipt shows seconds + amount.
4. **Budget cap:** seed a low balance → server **auto-stops** and the relay opens at `max_on_seconds`.
5. **The core proof — connectivity gating:** mid-session, **pull the relay device's network/power link** → the on-board watchdog **opens the relay** (supply off) and the server marks the session **suspended, billing frozen**. Reconnect within the window → session **auto-resumes, relay closes again**, and the **offline gap is not billed**.

If that demo runs, ADR-0001 is validated on real hardware and the riskiest part of v1 is de-risked.

Stack (minimal — optimise for build speed, not production)

- **Backend:** FastAPI + Tortoise + **SQLite** (no postgres/Redis — in-memory connection registry, single process is fine for a POC).
- **Device link:** minimal **custom JSON-over-WebSocket** (connect, heartbeat, start, stop, status) — not OCPP.
- **The relay device:** an **actual relay** on a microcontroller/Raspberry Pi (ESP32 + relay module, or Pi + relay HAT) running a **throwaway "relay-agent"** we write for the POC — connects, heartbeats, honours remote start/stop, and **opens the relay on link loss** (the watchdog). *This is a POC stand-in for the firmware the device team will build to protocol 2.7 — not the production firmware.*
- **Client:** thin **Vite + React web** app (one or two screens). **Not** Capacitor, **not** app-store.
- **Auth:** hardcoded single demo user. **No Clerk.**
- **Payments:** a `POST /wallet/topup` that just credits the ledger. **No Razorpay.**

Explicit cut list (NOT in the POC)

Production firmware / custom PCB · native app · app store · Clerk · real Razorpay · invoices · admin console (seed via API/DB) · map + geolocation (use a machine **list**) · QR scanning · multi-operator / settlement · Redis · staging/prod infra · Aerich migrations (recreate the DB) · full test suite (only the ~4 session-engine tests below).

Day-by-day (10 working days, 1 builder + Claude)

Day	Deliverable
1	Scaffold: FastAPI + Tortoise/SQLite, models (<code>Machine</code> , <code>SupplySession</code> , <code>Wallet</code> , <code>WalletTransaction</code>), config. Vite/React shell.
2	Device WS protocol + in-memory registry + heartbeat + status. Relay-agent + hardware bring-up (wire the relay; connect, heartbeat, honour start/stop, open-on-link-loss).
3	Remote start/stop dispatch + ack → relay physically switches ; <code>SupplySession</code> lifecycle.
4	Server-side per-session ticker — accrue connected-on seconds; live session state.
5	Billing on stop (<code>rate</code> × <code>seconds</code>), wallet debit, receipt; <code>balance</code> = SUM(<code>ledger</code>); fake top-up.
6	Budget-derived <code>max_on_seconds</code> cap → auto-stop (relay opens).
7	Disconnect → suspend → freeze billing ; resume within 30-min window → relay auto-closes ; finalize past window .
8	Web client: machine list, Start/Stop, live timer + cost , wallet, session history/receipt.
9	Integrate; demo-script the 5 criteria (incl. pulling the physical link); edge cases (double-start, stop-while-disconnected).
10	Buffer, cleanup, short README + recorded demo.

Key POC tests (the only ones worth writing now)

- Start→accrue→stop bills `rate` × `seconds` and debits the wallet.
- Budget cap auto-stops at `max_on_seconds` .
- Disconnect freezes billing; the off-gap is excluded from billable seconds.
- Reconnect inside the window resumes; outside it finalizes.

What the POC deliberately leaves unproven

Production firmware robustness, native/app-store UX, real payment flow, formal invoicing, scale/concurrency of the per-session ticker, and **zero-supply blindness** (still unmeasurable — a product-acceptance question, not something a POC can resolve). These stay in `PROJECT-PLAN.md` §1.

Timed Supply Control System (working name)

A greenfield product, architecturally modelled on the VoltLync EV-CSMS but for a **generic on/off supply billed by time, not energy**. A customer finds a **Machine** on a map (or scans its QR), starts it from the app, the Machine delivers an abstract **safe low-voltage supply** while switched on, and is billed for the **seconds the supply was on**. Unattended; pure-time (no flow sensor).

This is **not** the EV product and does **not** reuse its code — the EV CSMS is the *reference architecture* only. Terminology below is deliberately distinct from the EV glossary (`../.. /CONTEXT.md`) to prevent conflation.

Language

Machine: The customer-facing unit that delivers the supply, identified by a stable device id. Analogue of the EV **Charger/EVSE**, but a **Machine** has no energy meter — it is a relay plus a connection watchdog. Customer-facing label (per the concept wireframe: "search for machine", "picture of machine"). *Avoid:* "charger", "EVSE" (energy-metered connotation), "device" as a customer-facing word.

Supply: The abstract resource switched on/off behind the Machine's relay. **Safe low-voltage** (inherently safe to switch and to restart unattended). *Avoid:* "energy", "charge", "power" (imply metering that does not exist here).

Supply Session / Session: One billable episode from Start to Stop, billed by **connected-on seconds**. A Session may span **multiple connected-on intervals** with off-gaps between them (see **Connectivity-gated supply + Resume window**). Analogue of the EV **Charging Session**, but the billable quantity is time, not kWh. *Avoid:* "transaction" as the customer-facing word; "charging session".

Connected-on time / Billable seconds: The billable quantity: the sum of wall-clock seconds during which the Session was **both switched on and the Machine was connected to the server**. Measured **server-side**; there is no device-side counter. This is the metering primitive that replaces the EV system's device-reported kWh. *Avoid:* "duration" unqualified (the calendar span start→stop includes unbilled off-gaps and is a different number).

Connectivity-gated supply (the core invariant): *Supply flows only while the Machine is connected to the server*. On loss of the server link the Machine's watchdog opens the relay (supply OFF); on reconnect within the **Resume window** the supply auto-restarts (safe because the supply is low-voltage and unattended). Consequences: no unbilled/unbounded offline supply, the stop/budget command can always reach an actively-supplying Machine, and not billing an off-gap is *exact* (nothing flowed), not an approximation. See `[[adr-0001-connectivity-gated-server-metered-supply]]`. *Avoid:* assuming supply can be on while disconnected.

Resume window: The 30-minute grace after a disconnect during which a suspended Session may resume (accumulating more connected-on time) rather than being finalized. Analogue of the EV resume-staleness threshold (ADR 0022), but here it must sit *well above* the heartbeat interval so the ordering can never invert (the bug that force-finalized live EV resumes). *Avoid:* "timeout" unqualified.

Max-on cap / Budget-derived cap: The server-enforced upper bound on a prepaid Session's connected-on seconds: $\text{max_on_seconds} = \text{spendable_budget} / \text{rate_per_second}$. Enforced server-side by issuing a stop when elapsed connected-on time crosses it — always deliverable because an actively-supplying Machine is, by the invariant, connected. Analogue of the EV **Budget cap**, denominated in seconds instead of kWh.

Zero-supply blindness (a known, accepted limitation): Because billing is **pure-time with no sensor**, the system *cannot detect* that the supply behind the relay was dead — it bills for on-time regardless of whether anything actually flowed. The EV system's "zero-energy → full refund" path has **no analogue here** and cannot be built without adding a sensor. A decision the client must explicitly accept. See [[adr-0001-connectivity-gated-server-metered-supply]]. *Avoid:* promising any "nothing-flowed refund".

Relationships

- A **Supply Session** is metered by the server as accumulated **Connected-on time**; there is no device meter.
- **Connectivity-gated supply** makes the off-gap between two connected-on intervals unbillable *by construction* (supply was physically off).
- The **Max-on cap** is always enforceable because supply implies connectivity.

Supply is connectivity-gated and metered server-side by wall-clock time

For the Timed Supply product we bill by **seconds the supply is on**, not by a device-reported meter. We decided the **Machine may deliver supply only while it is connected to the server** (its watchdog opens the relay on link loss), and the **server** measures billable time on its own wall-clock as accumulated **connected-on seconds**. There is no device-side counter and no flow sensor.

Context — time has no meter, and that changes everything

The reference EV-CSMS bills `energy_consumed_kwh × rate`, where energy is a **monotonic counter reported by the device** via OCPP `MeterValues`. That single property silently underwrites three behaviours: the budget cap self-heals (a lost `RemoteStop` just re-fires next frame, energy only goes up), "zero-energy → full refund" is *measurable* (the meter proves nothing flowed), and the system never bills for supply it did not deliver (the meter, not the clock, is truth).

Billing by **time** comes with **no meter**. So the source of truth for billable seconds is a genuine fork with three shapes:

- **A — Server wall-clock.** `stop – start` measured server-side; device is a dumb relay. Simple, but a device that drops offline while physically ON is invisible to the server: keep the clock running (**overbill** across a dead link) or freeze it (**underbill**), and — worse — the server's only lever (`stop`) cannot reach an offline relay, so a wedged unit delivers **unbilled, unbounded supply** with nothing able to stop it. A safety hole, not just a billing one.
- **B — Device-reported on-duration.** The device counts its own on-time like a meter. Restores self-healing, but requires an RTC/counter and reporting protocol on the device.
- **C — Hybrid.** Server pushes a budget-derived max-on to the device; device self-enforces it and reports actual on-time on reconnect. Most robust, most work on both sides.

Decision

Take a **fourth path that makes A safe by construction**: gate the supply on connectivity.

- The **Machine** is a relay + a **connection watchdog**. On loss of the server link the watchdog **opens the relay** (supply OFF). No device counter, no RTC, no device-side metering — the simplest possible firmware.
- The **server** meters billable time on its wall-clock, accumulating only **connected-on seconds** (elapsed while the Session is on *and* the Machine is connected).
- The load-bearing invariant: **supply flows only while connected**.

Everything good follows from that one invariant:

1. **No unbilled/unbounded offline supply.** A dead link means the supply is *physically off*. The safety hole in path A is closed without a device-side fail-safe timer.

2. **The stop / budget cap is always enforceable.** An actively-supplying Machine is, by definition, connected, so a stop command can always reach it while it matters. (In the EV system a stop could be lost to an offline charger; here it cannot, *while supply is flowing*.)
3. **Not billing an off-gap is exact, not a concession.** Supply genuinely did not flow during a disconnect, so excluding that time is correct — the underbill worry of path A/A2 evaporates.
4. **Sessions span multiple connected-on intervals.** On reconnect within the **Resume window** (30 min, chosen $\gg$ heartbeat so the ordering cannot invert the way it did on the EV fleet), the supply **auto-restarts** and the Session accumulates more connected-on time. Auto-restart is acceptable **only because the supply is safe low-voltage and the deployment is unattended** — for a hazardous or attended supply this decision would flip to require an explicit human Start.

Accepted limitation — zero-supply blindness

Because metering is **pure-time with no sensor**, the system **cannot detect** that the resource behind the relay was dead. It bills for connected-on time whether or not anything actually flowed. The EV "zero-energy $\rightarrow$ full refund" path has **no analogue** and cannot exist without adding a sensor. This is a **product decision the client must explicitly accept**: the customer pays for on-time even if the supplied thing was broken. Revisiting it means adding a flow/energy sensor and a reconciliation path — a different (B/C-shaped) architecture.

Considered alternatives

- **A — pure server wall-clock, dumb relay (no connectivity gate).** Rejected: the offline-while-on case forces an overbill/underbill choice *and* leaves an uncontrollable, unbilled, potentially unsafe supply during a dead link. The connectivity gate is what rescues A.
- **B — device-reported on-duration.** Rejected for v1: pushes an RTC, a counter, and a reporting protocol onto the device for a benefit (billing across deliberate offline operation) this product does not need, since we *want* supply off when disconnected anyway.
- **C — server-pushed max-on + device self-enforcement + reported on-time.** Was briefly adopted mid-design then reverted: it is the right answer *only if* supply must continue across disconnects. Given the unattended, safe-low-voltage, supply-off-when-disconnected posture, the connectivity gate delivers the same safety with dumber, cheaper firmware.

Consequences

- **The server needs an active per-Session clock**, unlike the EV system whose budget checks are purely reactive to inbound `MeterValues` frames. Something must drive elapsed-time evaluation and the max-on cap on a timer (a server-side ticker or a heartbeat-driven check); the exact mechanism is an open build decision.
- **Heartbeat is the metering signal.** Billable time is bounded by the last proof-of-life, so heartbeat interval sets both metering granularity and how fast a disconnect is noticed. The Resume window (30 min) must stay well above it; a startup invariant check should assert `resume_window  $\gg$  heartbeat` so the EV ordering bug cannot recur.

- **Flaky links cause supply flap** (off on drop, auto-on on reconnect). Harmless for safe low-voltage; would need debounce/interlock for anything with mechanical inrush — another reason this ADR is scoped to the safe-low-voltage envelope.
- **The firmware is delivered by a separate team** against our frozen device WS protocol: relay + connection watchdog with a bounded open-on-link-loss timeout, heartbeat, and remote start/stop. No metering firmware required. The protocol is the interface contract; the watchdog behaviour is the one non-negotiable clause.